HARNESS LAB · MÓDULO 3 · AULA 3.4

Subagente: *quando usar*

O valor dele é a janela de contexto limpa, não a persona. Um subagente que tira o barulho da conversa principal vale mais que cinco cargos de mentira.

`.claude/agents/` `tools:` `model: haiku` `@agent-test-runner` `PreToolUse` `/bc-harness:ai-context` `--adopt`

VOCÊ SAI COM

O `test-runner` e o `revisor` no `.claude/agents/` do seu projeto, testados, e o `/ai-context` do BC Harness instalado para documentar o projeto com três subagentes.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A aula abre com o aviso mais importante: **quase nunca é útil usar subagente**. O time de personas (analista, arquiteto, dev, revisor) é conto de fadas. Numa tarefa de desenvolvimento ele só aumenta a complexidade e a sua carga cognitiva. Dá para rodar cada fase da implementação sem um subagente para cada cargo.

Ele se paga quando o papel é muito específico. O exemplo é o **test-runner**: dispara a suíte, vê o que falhou, diagnostica e sobe só o necessário para o agente principal. Economia de token e de contexto.

O que ele é de verdade

Uma estratégia de contexto. Começa com a janela vazia, só com o pedido do orquestrador, e devolve só o resultado. O caminho fica no contexto dele.

Por que é útil

Contexto limpo, trabalho em paralelo, limite de ferramenta imposto (só Read, Grep, Glob: não escreve nem edita) e o modelo certo: orquestrador no Opus 5.5, test-runner no Haiku.

Quando usar

A tarefa gera muita saída que você não vai usar depois: busca, log, teste. O Lucas usa para pesquisa de conteúdo do blog: vários subagentes leem páginas e um consolida.

Quando não

Na tarefa de desenvolvimento em si. Orquestrar é complicado: pode chamar o agente errado. Com uma boa especificação, o Claude Code vai do zero ao código sozinho. Já vêm prontos o Plan e o general-purpose.

> *"Use one when a side task would flood your main conversation with search results, logs, or file contents you won't reference again."*
>
> Documentação oficial do Claude Code · Create custom subagents (code.claude.com/docs/en/sub-agents)

A lição de casa: **construa o seu test-runner** e mande nos comentários da aula. Depois a aula mostra uma orquestração que faz sentido, do BC Harness: o `/ai-context`. É um comando, ou seja, um prompt que você chama e que dispara subagentes para documentar o projeto em 10 arquivos canônicos (overview, arquitetura, stack, code guidelines, regras de domínio, contratos de API, modelagem de dados, dependências, mais o `AGENTS.md` e o `CLAUDE.md`). Documentação viva: só o estado atual, nada de histórico de decisão, porque o histórico compete pela atenção do modelo.



O **inspector** é o único que lê o código inteiro (a aula diz que o `/ai-context` decide, pelo tamanho do repositório, se roda um ou vários; na versão 0.2.0 do plugin, ele dispara um inspector só). **Core** e **docs** recebem só o consolidado e escrevem sem degradar a qualidade: é o lost in the middle e o context rot do Módulo 1 virando separação de funções. O `--adopt` deixa o comando assumir um `CLAUDE.md` ou `AGENTS.md` que ele não gerou, como o do Laravel Boost.

295 linhas no `AGENTS.md` gerado na gravação, juntando o `/ai-context` e o bloco do Laravel Boost, com os ponteiros para `docs/agents/`. Medido na aula, não é número oficial. A doc do Claude Code recomenda menos de 200 linhas por arquivo de instruções.

2 Indo *além da aula*

ALÉM DA AULA · "VAZIO" QUER DIZER SEM A CONVERSA

O subagente não vê o seu histórico, as skills que você já chamou nem os arquivos já lidos. Mas um subagente seu (e o general-purpose) começa com o **CLAUDE.md de todos os níveis e o git status**. Só Explore e Plan começam sem o CLAUDE.md. Regra que precisa chegar a ele e não está no CLAUDE.md vai escrita no pedido. A exceção é o fork (`/subtask`): ele herda a conversa inteira.

Fonte: sub-agents, "What loads at startup": "Each subagent starts with a fresh, isolated context window."

ALÉM DA AULA · BASH ABRE A PORTA

Sem Edit e Write, mas com Bash, o test-runner ainda pode escrever arquivo (`sed -i`, `>`). Só leitura de verdade é o revisor, que não tem Bash. Para dar Bash com trava, use um hook `PreToolUse` no frontmatter do subagente, que só vale enquanto ele roda: é o `bash-so-permitido.sh` que a aula instala, usado pelas variantes em `modelos/`. Esse hook só roda depois que você aceita a confiança da pasta, e subagente que vem de plugin ignora o campo `hooks`.

Fonte: sub-agents, "Conditional rules with hooks" e "Hooks in subagent frontmatter". Hook testado com JSON falso: o que passa, o que bloqueia, JSON ilegível.

ALÉM DA AULA · GLOB E GREP NO LINUX, MACOS E WSL

Nessas plataformas o Claude Code tira Glob e Grep do conjunto padrão e busca com `find` e `grep` pelo Bash. Eles voltam para o subagente que os lista em `tools` **e deixa o Bash de fora**. Por isso o revisor (`Read, Grep, Glob`) tem as três, e o `Bash, Read, Grep` que aparece em muito exemplo de test-runner vira, na prática, `Bash, Read`. No teste, o test-runner pediu Glob e recebeu "No such tool available: Glob". O arquivo instalado já lista só `Bash, Read`. O mesmo vale para o inspector do BC Harness (`Read, Glob, Grep, Bash`): ele busca pelo Bash, e o "só leitura" é pedido no prompt, não imposto.

Fonte: tools-reference, "Glob tool behavior". Testado no Claude Code 2.1.316, WSL.

ALÉM DA AULA · ELE RODA EM SEGUNDO PLANO E PEDE PERMISSÃO NA SUA SESSÃO

Sem configurar nada, o subagente começa em background e o resultado chega numa notificação. Quando ele precisa de permissão, a pergunta aparece na sua sessão com o nome dele. Comando só de leitura (`ls`, `cat`, `grep`) roda sem perguntar. Uma regra como `Bash(npm test *)` aprova só o subcomando `npm test`: em `npm install && npm test`, o `npm install` ainda pede permissão. Em `claude -p` não há quem responda, e toda pergunta vira negação. Por isso a versão instalada lê os arquivos com Read e roda só o comando de teste, sozinho.

Fonte: sub-agents, "Run subagents in foreground or background"; permissions, "Compound commands" e "Read-only commands".

ALÉM DA AULA · O AGENTS.MD DO /AI-CONTEXT NÃO ENTRA SOZINHO

O `CLAUDE.md` que o `/ai-context` gera é um aviso de até 400 bytes com um link para o `AGENTS.md`, não um `@AGENTS.md`. E com `CLAUDE.md` no projeto, o Claude Code lê **só o CLAUDE.md**: o `AGENTS.md` só entra se o modelo decidir abrir. Para carregar os dois sempre: `/config`, **Project instructions**, `claude-md-and-agents-md`. Vale só para você (fica no `~/.claude/settings.json`), então cada dev do time faz o mesmo. Não acrescente o import à mão: o banner do arquivo avisa que edição manual é sobrescrita na próxima rodada.

Fonte: memory, "AGENTS.md" e "Choose which instruction files load"; `agents/ai-context-core.md` do BC Harness.

ALÉM DA AULA · GARANTIR QUAL SUBAGENTE RODA, E QUANDO REINICIAR

"Use o test-runner" deixa a decisão com o Claude. `@agent-test-runner` no pedido garante que ele roda. Arquivo novo ou editado em `.claude/agents/` vale em segundos, sem reiniciar, **se a pasta já existia** quando a sessão abriu. Criou a pasta agora? Reinicie. E arquivo sem `description` é pulado sem aviso na sessão; o `claude plugin validate .claude/agents` mostra isso só como warning e termina com "Validation passed with warnings", com saída 0.

Fonte: sub-agents, "Invoke subagents explicitly", "Write subagent files" e "Subagent files Claude Code skips". O validate foi testado com arquivo sem description.

3 Instale e faça *no seu repositório*

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Instale os dois subagentes e o hook no seu projeto.

Cria `.claude/agents/test-runner.md`, `.claude/agents/revisor.md` e `.claude/hooks/bash-so-permitido.sh`. Se você já tem um arquivo com o mesmo nome, o seu fica intacto e o da aula vai para `.harness-lab-novo`.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 3.4
```

3 Confira o frontmatter.

O esperado é "Validation passed", sem o "with warnings". Warning de `description` quer dizer que o Claude Code vai pular o arquivo.

Terminal · na raiz do seu projeto — shell

```
ls .claude/agents
claude plugin validate .claude/agents
```

4 Prove que o hook trava, sem abrir o Claude Code.

Mesmo método da aula 2.4: JSON falso no stdin. O esperado é a mensagem "Bloqueado pelo hook do subagente" e `exit 2`. Troque o comando por `git diff --stat` e o hook sai com 0.

Terminal · na raiz do seu projeto — shell

```
H=.claude/hooks/bash-so-permitido.sh
j='{"tool_input":{"command":"git diff; rm -rf ."}}'
echo "$j" | $H "git diff" || echo "exit $?"
```

5 Faça a prática: a suíte dentro e fora do contexto.

Abra o Claude Code na raiz do projeto (reinicie se a pasta `.claude/agents/` acabou de ser criada). Primeiro peça "rode a suíte inteira e me diga o que falhou" sem subagente e olhe o `/context`. Depois `/clear` e:

Dentro do Claude Code — sessão

```
> @agent-test-runner rode a suíte inteira
/context
```

Quando ele pedir permissão para o comando de teste, aprove. No `/context`, as mensagens ficam menores: o log ficou na janela dele. Para o revisor, mande o diff no pedido, porque ele não tem Bash:

Dentro do Claude Code — sessão

```
> @agent-revisor revise o diff da branch. Mande o git diff no pedido
```

6

Instale o BC Harness, o plugin do /ai-context.

É o plugin com os três subagentes da aula (`ai-context-inspector`, `ai-context-core`, `ai-context-docs`) e o comando. Fica instalado para o seu usuário, em todos os projetos.

Terminal — claude

```
claude plugin marketplace add beerandcodeteam/beer-and-code-harness
claude plugin install bc-harness@beer-and-code
claude plugin list
```

Ou, dentro do Claude Code:

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/beer-and-code-harness
/plugin install bc-harness@beer-and-code
/reload-plugins
```

O plugin traz também `/bc-harness:init`, `/bc-harness:plan` e um hook de Bash (`sail-guard`): em projeto com `vendor/bin/sail`, ele bloqueia `php`, `artisan` e `composer` no host e devolve o comando com `./vendor/bin/sail`.

7

Rode o /ai-context na raiz do projeto real.

Com `--adopt` se você já tem `CLAUDE.md` ou `AGENTS.md` que ele não gerou (o do Laravel Boost, por exemplo). Sem ele, esses arquivos são pulados com o aviso "skipped (not owned)". Em projeto grande demora bastante: é normal, ele lê tudo.

Dentro do Claude Code · no seu projeto real — bc-harness

```
/bc-harness:ai-context --adopt
```

Ele não faz commit. Revise o que mudou e confira o que o Claude Code carrega:

Terminal · na raiz do seu projeto — shell

```
git status --short
git diff --stat
```

Dentro do Claude Code — sessão

```
/memory
/config
```

No `/memory`, veja se o `AGENTS.md` aparece. Se não, no `/config` mude **Project instructions** para `claude-md-and-agents-md` (seção 2).

8 **Opcional: o revisor que busca o diff sozinho.**

Troca o revisor instalado pela variante com Bash travado em `git diff`, `git log`, `git show`, `git status` e `grep`. Abra o Claude Code e aceite a confiança da pasta, senão o hook não roda. O comando troca o seu `revisor.md` atual: se ele é seu, guarde uma cópia antes.

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-3-regra/aula-3.4-subagente-quando-usar
cp "$A/modelos/revisor-com-git.md" .claude/agents/revisor.md
```

O test-runner *instalado*

É o ponto de partida da lição de casa (trecho levemente resumido). Ele descobre o comando pela stack. Para o seu projeto, o melhor é fixar: troque a lista por uma linha só, com o comando exato (`./vendor/bin/sail artisan test --compact`, `pnpm vitest run`, `pytest -q`, `bundle exec rspec`).

.claude/agents/test-runner.md — instalado pelo instalar.sh · não copie do PDF

```
---
name: test-runner
description: Roda os testes do projeto e devolve só as falhas resumidas. Use depois de uma mudança e quando
pedirem para rodar ou depurar testes.
tools: Bash, Read
model: haiku
---
Você roda testes e resume o resultado. Não corrige nada: quem corrige é a conversa principal, que tem o
contexto da mudança.

## 1. O comando
Use o comando que o pedido trouxer. Se não trouxer, use o do CLAUDE.md ou do AGENTS.md. Se nenhum dos dois
disser, descubra abrindo os arquivos da raiz com Read (não com o Bash), nesta ordem:
- vendor/bin/sail e um compose do Sail: ./vendor/bin/sail artisan test --compact
- arquivo artisan: php artisan test --compact
- package.json com script test: pnpm, yarn ou npm test, pelo lockfile
- pyproject.toml, pytest.ini ou tests/ com test_*.py: pytest -q
- Gemfile com rspec: bundle exec rspec · go.mod: go test ./...
Para fixar o comando deste projeto, troque a lista acima por uma linha só com ele.

Use o Bash só para rodar o comando de teste, sozinho: sem cd, sem &&, na raiz do projeto. Assim ele casa com a
regra de permissão do projeto (por exemplo Bash(npm test *)) e não para pedindo autorização.

## 2. A saída
Saída longa: rode o mesmo comando com | tail -60 ou | grep nas falhas.

## 3. A resposta (no máximo 15 linhas)
- o comando que rodou; passou ou falhou, e quantos testes;
- cada falha: arquivo:linha, a mensagem em uma linha e uma hipótese da causa.
```

```
Se o comando nem chegou a rodar os testes (container parado, dependência faltando), diga isso em uma linha, com o erro.
```

4 Arquivos que *reforçam a aula*

`.claude/agents/test-runner.md`

O subagente que se paga: saída enorme, conclusão curta, Haiku, sem Edit e Write. Testado numa sessão real: 32 testes, 1 falha, devolvida com `arquivo:linha` e hipótese.

No repositório: modulo-3-regra/aula-3.4-subagente-quando-usar/projeto/

`.claude/agents/revisor.md`

Olhos novos, mãos amarradas: `Read, Grep, Glob` e mais nada. O prompt manda ignorar a conclusão que vier no pedido, porque revisor que recebe a sua hipótese tende a confirmá-la.

No repositório: modulo-3-regra/aula-3.4-subagente-quando-usar/projeto/

`modelos/revisor-com-git.md + .claude/hooks/bash-so-permitido.sh`

O limite imposto levado a sério: o revisor ganha Bash, mas um hook no frontmatter só deixa passar leitura do git e `grep`. Junta a trava do Módulo 2 com o subagente.

No repositório: modulo-3-regra/aula-3.4-subagente-quando-usar/modelos/ e projeto/

`modelos/test-runner-travado.md`

O test-runner com o comando do Sail fixo e o Bash travado nele. Para quando você quer a garantia de que ele só roda teste, e não só o pedido no prompt.

No repositório: modulo-3-regra/aula-3.4-subagente-quando-usar/modelos/

`modelos/pedido-para-subagente.md`

O pedido é tudo que ele sabe: objetivo, o que já foi descartado, onde olhar e o formato da resposta. Com pedido ruim e bom lado a lado.

No repositório: modulo-3-regra/aula-3.4-subagente-quando-usar/modelos/

`agents/ai-context-inspector.md`

O prompt completo do inspector, para ler com calma: persona, só leitura, o que nunca ler (`.spec/`, `.env`, `vendor/`) e o formato do digest. Os de core e docs estão ao lado.

No repositório: github.com/beerandcodeteam/beer-and-code-harness (e, depois de instalar, ~/.claude/plugins/cache/beer-and-code/bc-harness/)

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
O Claude não acha o test-runner	A pasta `.claude/agents/` foi criada com a sessão aberta, ou o arquivo não tem `description` (é pulado sem aviso)	Reinicie o Claude Code. Rode `claude plugin validate .claude/agents` e trate warning como erro
O test-runner volta "permissão negada" e não roda nada	Ele encadeou com `&&` um comando que pede permissão (`npm install && npm test`) e a regra `Bash(npm test *)` não cobre o resto. Em `claude -p`, pergunta vira negação	Use o arquivo da aula, que lê com Read e roda só o comando de teste. Aprove a pergunta de permissão na sessão

SINTOMA	CAUSA	CORREÇÃO
"No such tool available: Glob" no subagente	Linux, macOS e WSL: com Bash na lista, Glob e Grep não voltam	Liste só o que existe (`Bash, Read`). Quer Glob e Grep? Tire o Bash
O revisor comenta o arquivo inteiro, não a mudança	Sem Bash ele não roda `git diff`; sem o diff no pedido, revisa o estado atual (no teste, tentou ler `.git/ORIG_HEAD`)	Mande o diff no pedido ou use `modelos/revisor-com-git.md`
O hook do frontmatter não bloqueia nada	Pasta não confiada, sessão `claude -p`, ou agente vindo de plugin	Abra o Claude Code na pasta e aceite a confiança. Para plugin, copie o agente para `.claude/agents/`
O `/ai-context` diz "skipped (not owned)"	O `CLAUDE.md` ou `AGENTS.md` não tem o banner dele	Rode de novo com `--adopt`: ele junta as suas regras e assume o arquivo
O `AGENTS.md` existe e o Claude ignora	Com `CLAUDE.md` no projeto, só ele carrega	`/memory` para ver; `/config`, Project instructions, `claude-md-and-agents-md`
A regra que você escreveu no `AGENTS.md` sumiu	Arquivo com o banner do `/ai-context` é sobrescrito na próxima rodada	Regra do time fora dele: `.claude/rules/` (aula 3.2) ou hook

6 Você sai *com*

- [ ] `claude plugin validate .claude/agents` passa sem warning no seu projeto.
- [ ] `@agent-test-runner rode a suíte inteira` devolve até 15 linhas e o `/context` encolhe.
- [ ] O revisor responde com `arquivo:linha`, o problema e a correção, sem editar nada.
- [ ] O hook bloqueia `git diff; rm -rf .` com `exit 2` no teste com JSON falso.
- [ ] `claude plugin list` mostra `bc-harness@beer-and-code`; o diff do `/ai-context` foi revisado.
- [ ] `.claude/agents/` no commit: o time recebe os mesmos subagentes.

PROMETIDO NA AULA

- ✓ **"Eu vou mostrar esses subagentes lá no BC Harness"**: o diagrama da seção 1 e o plugin no passo 6.
- ✓ **"Construa o seu test runner"**: a lição é sua. Ponto de partida: `projeto/.claude/agents/test-runner.md` (passo 2); travado: `modelos/test-runner-travado.md`.
- ✓ **"Isso vai estar aqui para vocês lerem por completo"** (o prompt do inspector): `agents/ai-context-inspector.md` no repositório do BC Harness, seção 4.
- ✓ **"Pegue esses três subagentes e o comando de AI Context e rode no seu projeto"**: plugin `bc-harness@beer-and-code`, passos 6 e 7.

7 Glossário

Subagente	Outra instância do modelo, com janela de contexto própria, prompt próprio e ferramentas próprias. Devolve só o resultado.
Conversa principal	A sua sessão. É o orquestrador: escreve o pedido para o subagente e recebe a resposta.
Pedido (handoff)	A mensagem que a conversa principal escreve ao delegar. Junto com o CLAUDE.md, é tudo que o subagente sabe da tarefa.
Fork	Subagente que herda a conversa inteira em vez de começar vazio. Você abre um com `/subtask`.
Frontmatter	O bloco entre `---` no topo do arquivo: `name`, `description`, `tools`, `model`, `hooks`.
Workspace trust	O diálogo de confiança da pasta. Sem aceitar, hook no frontmatter de subagente de projeto não roda.
AGENTS.md	Arquivo de instruções lido por vários agentes de código. O Claude Code lê quando não há CLAUDE.md, ou sempre, com `claude-md-and-agents-md`.
--adopt	Opção do `/ai-context` que assume um arquivo que ele não gerou, juntando as regras escritas à mão.

8 Para *ir além*

- Create custom subagents: frontmatter completo, "What loads at startup", background, fork, hooks no frontmatter e o limite de três níveis de subagente dentro de subagente.
- Tools reference: "Glob tool behavior", quando Glob e Grep existem no Linux, macOS e WSL.
- Memory: seção "AGENTS.md", a tabela do que carrega quando há CLAUDE.md e AGENTS.md.
- Best practices: o revisor com "only tools: Read, Grep, Glob", um limite que o Claude Code impõe, não o prompt.
- BC Harness: o README do `/ai-context` (os 10 artefatos, `+id`, `-id`, `--adopt`) e os prompts dos três agentes.

Próxima: *Aula 3.5 · Contexto na prática*

O subagente protege a sua janela do barulho. Mas e a sessão que já está cheia? Compactar, limpar ou começar de novo.